Ministry of Education of Republic of Moldova
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics

Laboratory work 5: Empirical Analysis of Dijkstra and Floyd–Warshall Algorithms

Author:
st. gr. FAF-241 Moisei Daniel
Verified:
asist. univ. Fiștic Cristofor

Chișinău - 2026

Contents

ALGORITHM ANALYSIS **2**
Objective 2
Tasks 2
Introduction 2

ALGORITHM IMPLEMENTATIONS **3**
Dijkstra (Classic) 3
Dijkstra (Optimized) 6
Floyd–Warshall (Classic) 9
Floyd–Warshall (Optimized) 11

EMPIRICAL ANALYSIS BY GRAPH DENSITY **14**
Very Sparse Graph 14
Sparse Graph 16
Dense Graph 18
Very Dense Graph 20

OVERALL RESULTS **22**

CONCLUSION **23**

REFERENCES **24**

ALGORITHM ANALYSIS

Objective

Study, implement, and empirically analyze the performance of four algorithm variants: classic and optimized versions of Dijkstra's shortest path algorithm and the Floyd–Warshall all-pairs shortest path algorithm, evaluated across four graph density levels and varying input sizes.

Tasks:

1. Study the dynamic programming method of designing algorithms;
2. Implement classic and optimized versions of Dijkstra and Floyd–Warshall using dynamic programming;
3. Perform empirical analysis of these algorithms for sparse and dense graphs;
4. Analyze how increasing the number of nodes influences the algorithms;
5. Present a graphical visualization of the collected data;
6. Compile a report summarizing the findings.

Introduction:

Shortest path computation is one of the most foundational problems in computer science and graph theory, with applications spanning network routing, navigation systems, logistics optimization, and compiler dependency resolution. In this laboratory, we implement and benchmark four algorithm variants — Dijkstra Classic, Dijkstra Optimized, Floyd–Warshall Classic, and Floyd–Warshall Optimized — across four structurally distinct graph densities to observe how implementation choices and graph structure jointly influence real-world performance.

Dynamic programming underlies both algorithms: Floyd–Warshall explicitly builds a solution table bottom-up by iteratively relaxing all pairs through intermediate vertices, while Dijkstra greedily selects the locally optimal unvisited vertex at each step, implicitly accumulating optimal substructure. While their theoretical complexities differ — $O((V+E)\log V)$ for optimized Dijkstra versus $O(V^3)$ for Floyd–Warshall — practical execution time is shaped by memory access patterns, data structure overhead, cache behavior, and the density of the input graph. By testing across very sparse, sparse, dense, and very dense graph forms, this study provides a comprehensive empirical picture of shortest path algorithm behavior beyond what asymptotic notation alone can reveal.

ALGORITHM IMPLEMENTATIONS

Dijkstra (Classic)

The classic Dijkstra implementation uses a linear scan over all unvisited vertices to find the minimum-distance node at each step. A `bool[]` array tracks visited vertices and a `double[]` array stores tentative distances from the source. This approach faithfully reflects the original algorithm as described by Dijkstra in 1959, prioritizing simplicity and correctness over raw performance.

Pseudocode:

Listing 1: Dijkstra Classic

```
Dijkstra(graph, source):
    dist    = double[graph.VertexCount]  // fill with infinity
    visited = bool[graph.VertexCount]
    dist[source] = 0

    for iter = 0 to VertexCount - 1:
        u = vertex with minimum dist[u] where not visited[u]   // O(V)
            scan
        if u == -1 or dist[u] == infinity: break
        visited[u] = true

        for each (neighbor v, weight w) in graph.GetNeighbors(u):
            newDist = dist[u] + w
            if newDist < dist[v]:
                dist[v] = newDist

    return dist
```

Implementation:

```
public static class DijkstraClassic
{
    public static double[] Run(IGraph graph, int source)
    {
        int n = graph.VertexCount;
        double[] dist = new double[n];
        Array.Fill(dist, double.PositiveInfinity);
        dist[source] = 0;

        bool[] visited = new bool[n];

        // Simple list-based "priority queue": always scan for minimum
        for (int iter = 0; iter < n; iter++)
        {
            // Find unvisited vertex with minimum distance (O(V) scan)
            int u = -1;
            for (int i = 0; i < n; i++)
            {
                if (!visited[i] && (u == -1 || dist[i] < dist[u]))
                    u = i;
            }
            if (u == -1 || double.IsPositiveInfinity(dist[u])) break;
            visited[u] = true;

            foreach (var (v, w) in graph.GetNeighbors(u))
            {
                double newDist = dist[u] + w;
                if (newDist < dist[v])
                    dist[v] = newDist;
            }
        }
        return dist;
    }
}
```

Figure 1: Dijkstra Classic implementation in C#

Complexity Facts & Performance Characteristics:

- **Complexity:** $O(V^2)$ time, $O(V)$ space.
- **Linear scan overhead:** Finding the minimum-distance unvisited vertex requires iterating over all V vertices at each of the V iterations, producing a V^2 inner loop that dominates on large graphs. This becomes especially costly on dense graphs where many vertices remain unvisited for a long time.
- **Simplicity advantage:** On very dense graphs where $E \approx V^2$, the $O(V^2)$ classic version can actually match or outperform the heap-based optimized version, since the heap operations themselves become expensive when the priority queue is flooded with many entries.

- **No GC pressure:** The flat array structure avoids heap allocations beyond the initial setup, keeping garbage collection interference minimal throughout execution.

Dijkstra (Optimized)

The optimized Dijkstra replaces the $O(V)$ linear scan with a binary min-heap implemented via .NET's built-in `PriorityQueue<TElement, TPriority>` introduced in .NET 6. Rather than scanning all vertices, the heap always provides the minimum-distance unvisited vertex in $O(\log V)$ time. Stale heap entries — vertices re-inserted after a distance improvement — are discarded lazily using a distance comparison on extraction.

Pseudocode:

Listing 2: Dijkstra Optimized (Binary Min-Heap)

```
Dijkstra(graph, source):
    dist = double[graph.VertexCount]  // fill with infinity
    dist[source] = 0
    pq = PriorityQueue<vertex, distance>
    pq.Enqueue(source, 0.0)

    while pq not empty:
        (u, d) = pq.Dequeue()
        if d > dist[u]: continue   // stale entry, skip

        for each (neighbor v, weight w) in graph.GetNeighbors(u):
            newDist = dist[u] + w
            if newDist < dist[v]:
                dist[v] = newDist
                pq.Enqueue(v, newDist)

    return dist
```

Implementation:

```csharp
public static class DijkstraOptimized
{
    public static double[] Run(IGraph graph, int source)
    {
        int n = graph.VertexCount;
        double[] dist = new double[n];
        Array.Fill(dist, double.PositiveInfinity);
        dist[source] = 0;

        // PriorityQueue<(vertex), (priority=distance)>
        var pq = new PriorityQueue<int, double>(n);
        pq.Enqueue(source, 0.0);

        while (pq.Count > 0)
        {
            pq.TryDequeue(out int u, out double d);

            // Stale entry: a shorter path was already found
            if (d > dist[u]) continue;

            foreach (var (v, w) in graph.GetNeighbors(u))
            {
                double newDist = dist[u] + w;
                if (newDist < dist[v])
                {
                    dist[v] = newDist;
                    pq.Enqueue(v, newDist);
                }
            }
        }

        return dist;
    }
}
```

Figure 2: Dijkstra Optimized implementation in C#

Why the Optimized Version is Faster:

- **Binary heap extraction:** Replacing the $O(V)$ linear scan with an $O(\log V)$ heap extraction reduces the overall complexity from $O(V^2)$ to $O((V + E)\log V)$. This is especially significant on sparse graphs where $E \ll V^2$, as far fewer relaxations occur and the heap stays small.

- **Lazy deletion of stale entries:** When a shorter path is found for a vertex already in the queue, a new entry is pushed rather than updating the existing one. Stale entries are identified and discarded on extraction via a simple distance comparison. This avoids the need for a decrease-key operation and keeps the implementation simple while preserving correctness.

- **Classic vs Optimized:** On sparse and medium-density graphs, the optimized version delivers substantial speedups. On very dense graphs approaching $O(V^2)$ edges, the heap accumulates many entries and the $O(\log V)$ overhead per operation can erode the advantage, sometimes making the classic version competitive at large sizes.

Floyd–Warshall (Classic)

Floyd–Warshall is a dynamic programming algorithm that computes the shortest paths between all pairs of vertices simultaneously. It works by iteratively considering each vertex k as a potential intermediate node and relaxing all pairs (i, j) through k. The classic implementation operates directly on a two-dimensional `double[,]` adjacency matrix, making the algorithm straightforward to implement and reason about.

Pseudocode:

Listing 3: Floyd–Warshall Classic

```
FloydWarshall(distMatrix):
    d = copy of distMatrix        // d[i,j] = weight of edge i->j, 0 on
        diagonal, infinity if no edge

    for k = 0 to n - 1:
        for i = 0 to n - 1:
            for j = 0 to n - 1:
                via = d[i,k] + d[k,j]
                if via < d[i,j]:
                    d[i,j] = via

    return d
```

Implementation:

```
public static class FloydWarshallClassic
{
    public static double[,] Run(double[,] dist)
    {
        int n = dist.GetLength(0);
        // Work on a copy so the original is not mutated
        double[,] d = (double[,])dist.Clone();

        for (int k = 0; k < n; k++)
            for (int i = 0; i < n; i++)
                for (int j = 0; j < n; j++)
                {
                    double via = d[i, k] + d[k, j];
                    if (via < d[i, j])
                        d[i, j] = via;
                }

        return d;
    }
}
```

Figure 3: Floyd–Warshall Classic implementation in C#

Complexity Facts & Performance Characteristics:

- **Complexity:** $O(V^3)$ time, $O(V^2)$ space.
- **Dynamic programming structure:** The outermost loop over k defines the DP stage. After iteration k, `d[i,j]` contains the shortest path from i to j using only vertices $\{0, 1, \ldots, k\}$ as intermediates. This optimal substructure property guarantees correctness.
- **2D array access pattern:** The standard two-dimensional array `d[i,k]` and `d[k,j]` cause row-major access violations in the innermost loop. For each fixed k and i, `d[k,j]` strides across consecutive memory addresses (cache-friendly), but `d[i,k]` is a fixed scalar that .NET may not hoist out of the loop, creating redundant lookups.
- **No graph density dependence:** Unlike Dijkstra, Floyd–Warshall's runtime is entirely determined by V and is independent of E. This makes it equally expensive on sparse and dense graphs of the same vertex count.

Floyd–Warshall (Optimized)

The optimized Floyd–Warshall introduces three targeted improvements over the classic version: the 2D matrix is flattened to a 1D `double[]` array for better cache-line utilization, the `d[i,k]` value is cached in a scalar variable before the inner loop to eliminate redundant 2D lookups, and an early-exit condition skips the entire inner loop when `d[i,k]` is infinity — meaning no path from i to k exists and no relaxation through k is possible.

Pseudocode:

Listing 4: Floyd–Warshall Optimized (1D Flat Array + Row Cache + Early Exit)

```
FloydWarshall(distMatrix):
    n = distMatrix.rows
    d = flatten distMatrix to 1D array   // d[i*n + j]

    for k = 0 to n - 1:
        for i = 0 to n - 1:
            dik = d[i*n + k]
            if dik == infinity: continue   // early exit: no path i->k

            rowI = i * n
            rowK = k * n

            for j = 0 to n - 1:
                via = dik + d[rowK + j]
                if via < d[rowI + j]:
                    d[rowI + j] = via

    return reshape d back to 2D
```

Implementation:

```
public static class FloydWarshallOptimized
{
    public static double[] Run(double[,] distMatrix)
    {
        int n = distMatrix.GetLength(0);

        // Flatten to 1D: index [i*n + j]
        double[] d = new double[n * n];
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                d[i * n + j] = distMatrix[i, j];

        for (int k = 0; k < n; k++)
        {
            for (int i = 0; i < n; i++)
            {
                double dik = d[i * n + k];
                if (double.IsPositiveInfinity(dik)) continue; // early exit

                int rowI = i * n;
                int rowK = k * n;

                for (int j = 0; j < n; j++)
                {
                    double via = dik + d[rowK + j];
                    if (via < d[rowI + j])
                        d[rowI + j] = via;
                }
            }
        }

        return d;
    }

    /// <summary>Convenience method that reshapes the flat result back to 2D.</summary>
    public static double[,] RunMatrix(double[,] distMatrix)
    {
        int n = distMatrix.GetLength(0);
        double[] flat = Run(distMatrix);
        double[,] result = new double[n, n];
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                result[i, j] = flat[i * n + j];
        return result;
    }
}
```

Figure 4: Floyd–Warshall Optimized implementation in C#

Why the Optimized Version is Faster:

- **Flat 1D array layout:** A `double[n*n]` array places all matrix elements in a single contiguous memory block. This allows the CPU prefetcher to load entire rows into cache lines in a single pass, whereas the 2D `double[,]` representation adds an extra indirection layer and padding that disrupts sequential access.

- **Row index caching:** Computing `rowI = i*n` and `rowK = k*n` once per i-iteration and reusing them across the j-loop eliminates the two multiplications per inner iteration that the classic version performs implicitly via 2D indexing.

- **Early exit on infinity:** When `d[i,k]` is infinite, no path from i to any other vertex can be improved by routing through k. Skipping the entire j-loop in this case provides significant savings on sparse graphs where many such entries exist.

- **Scalar caching of `dik`:** Storing `d[i*n+k]` in a local variable before the inner loop allows the JIT compiler to keep it in a register throughout the j iteration, avoiding repeated memory reads of the same value.

EMPIRICAL ANALYSIS BY GRAPH DENSITY

Each subsection below presents benchmark results (averages of 5 runs) for one graph density level, covering Dijkstra on sizes from 100 to 3000 vertices and Floyd–Warshall on sizes from 100 to 500 vertices (due to its cubic complexity). A terminal output figure and two performance plots — one per algorithm family — are included for each density. Graph density is controlled by an edge probability factor: Very Sparse uses 5% of possible edges, Sparse uses 30%, Dense uses 70%, and Very Dense uses 95%, approximating a complete graph.

Very Sparse Graph

A graph with approximately 5% of the maximum possible edges, where most vertices have very few connections. This density level models real-world networks such as road maps and social graphs where connections are rare relative to the population size.

Results:

```
VerySparse    2000  DijkstraClassic   8.228 ms
VerySparse    2000  DijkstraOptimized 5.856 ms
VerySparse    2500  DijkstraClassic   10.433 ms
VerySparse    2500  DijkstraOptimized 5.041 ms
VerySparse    3000  DijkstraClassic   15.968 ms
VerySparse    3000  DijkstraOptimized 6.470 ms
```

Figure 5: Very Sparse — Dijkstra Terminal Benchmark Output

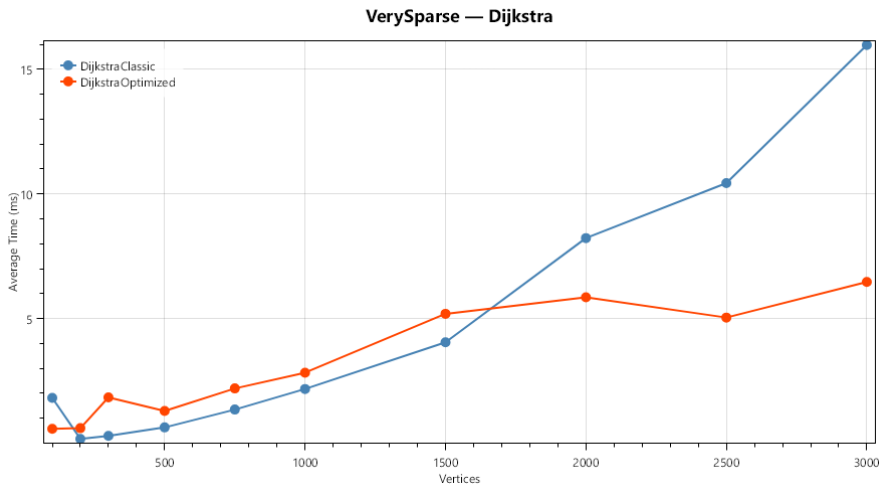

Figure 6: Very Sparse — Dijkstra Performance Plot

```
VerySparse     400  FloydWarshallClassic   115.085 ms
VerySparse     400  FloydWarshallOptimized 50.159 ms
VerySparse     450  FloydWarshallClassic   163.409 ms
VerySparse     450  FloydWarshallOptimized 69.677 ms
VerySparse     500  FloydWarshallClassic   225.682 ms
VerySparse     500  FloydWarshallOptimized 92.173 ms
```

Figure 7: Very Sparse — Floyd–Warshall Terminal Benchmark Output

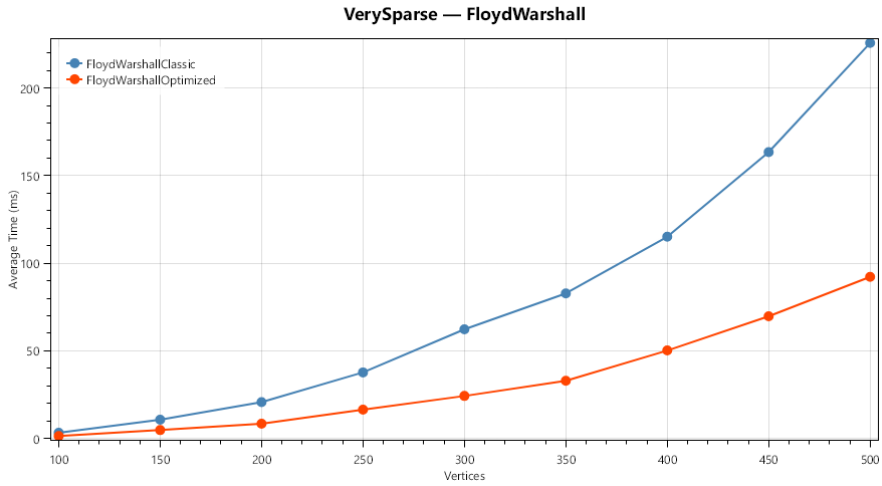

Figure 8: Very Sparse — Floyd–Warshall Performance Plot

Analysis: On very sparse graphs, Dijkstra Optimized demonstrates its most decisive advantage over the classic variant. With $E \approx 0.05 \cdot \frac{V(V-1)}{2}$, the priority queue remains small throughout execution and each heap operation processes only a handful of neighbors, keeping the $O(\log V)$ cost minimal. Dijkstra Classic's $O(V^2)$ linear scan dominates the runtime even when there are few edges to relax, since it must scan all vertices regardless of edge count. Floyd–Warshall behaves identically on very sparse and very dense graphs of the same vertex count, confirming its density-independence: the $O(V^3)$ cubic growth is clearly visible even in the 100–500 vertex range, with times growing dramatically. The early-exit optimization in Floyd–Warshall Optimized provides its greatest benefit here, since many `d[i,k]` entries remain infinity when edge density is low, allowing large portions of the inner loop to be skipped entirely.

Sparse Graph

A graph with approximately 30% of the maximum possible edges. This density level represents intermediate connectivity such as communication networks or dependency graphs where vertices have a moderate number of links but the graph remains far from complete.

Results:

```
Sparse        2000  DijkstraClassic   8.727 ms
Sparse        2000  DijkstraOptimized 6.455 ms
Sparse        2500  DijkstraClassic   14.069 ms
Sparse        2500  DijkstraOptimized 10.057 ms
Sparse        3000  DijkstraClassic   19.701 ms
Sparse        3000  DijkstraOptimized 15.315 ms
```

Figure 9: Sparse — Dijkstra Terminal Benchmark Output

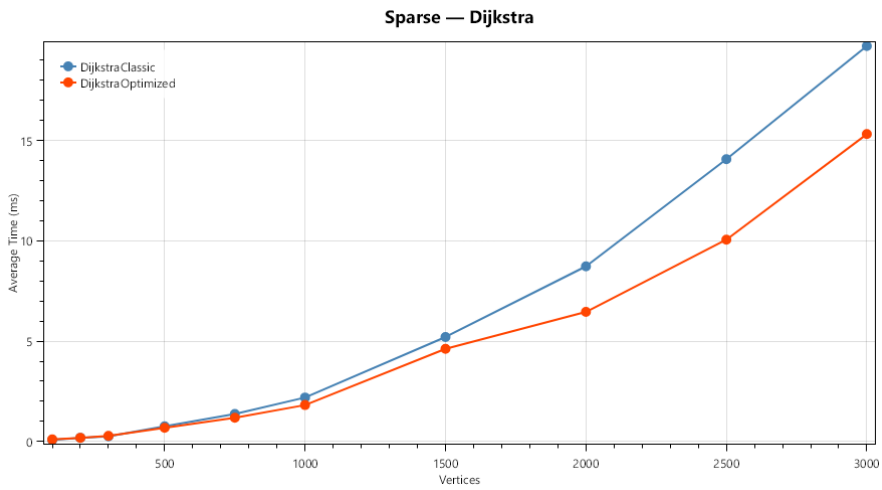

Figure 10: Sparse — Dijkstra Performance Plot

```
Sparse        400  FloydWarshallClassic   120.041 ms
Sparse        400  FloydWarshallOptimized 57.353 ms
Sparse        450  FloydWarshallClassic   158.920 ms
Sparse        450  FloydWarshallOptimized 82.736 ms
Sparse        500  FloydWarshallClassic   215.295 ms
Sparse        500  FloydWarshallOptimized 100.317 ms
```

Figure 11: Sparse — Floyd–Warshall Terminal Benchmark Output

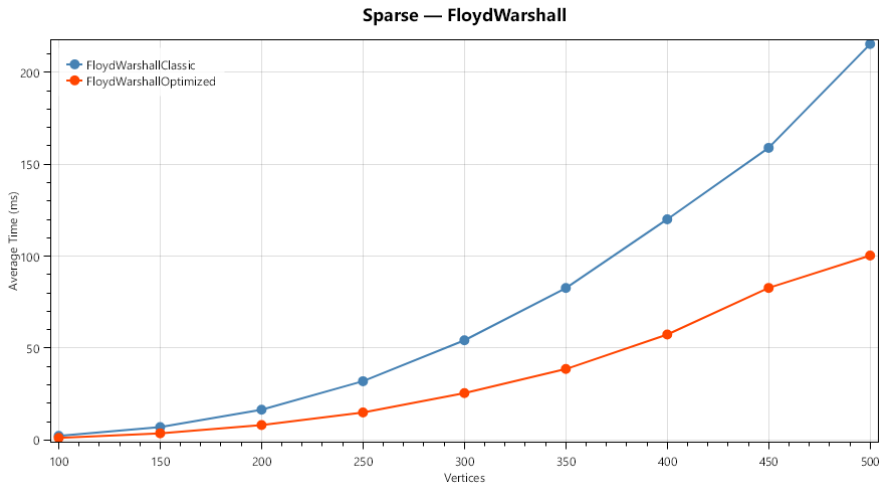

Figure 12: Sparse — Floyd–Warshall Performance Plot

Analysis: At 30% density, the priority queue in Dijkstra Optimized begins to grow more substantially during execution, as more relaxations occur and more entries are pushed per vertex. Nevertheless, the heap-based approach maintains a clear advantage over the classic linear scan, as E is still considerably less than V^2. Dijkstra Classic's runtime grows more steeply than at very sparse density, reflecting the increased number of relaxation steps in the inner loop. For Floyd–Warshall, the sparse density reduces the benefit of the early-exit optimization compared to the very sparse case: more `d[i,k]` entries are finite, meaning more inner j-loops must execute. The flat-array and row-cache optimizations continue to provide consistent savings, and Floyd–Warshall Optimized maintains a measurable lead over its classic counterpart throughout the tested vertex range.

Dense Graph

A graph with approximately 70% of the maximum possible edges, representing highly interconnected networks such as neural connectivity maps or financial correlation graphs. At this density, each vertex connects to the majority of other vertices, significantly increasing the number of relaxations Dijkstra must process and reducing the effectiveness of early-exit conditions in Floyd–Warshall.

Results:

```
Dense          2000  DijkstraClassic   15.079 ms
Dense          2000  DijkstraOptimized 12.731 ms
Dense          2500  DijkstraClassic   24.218 ms
Dense          2500  DijkstraOptimized 18.961 ms
Dense          3000  DijkstraClassic   31.377 ms
Dense          3000  DijkstraOptimized 26.520 ms
```

Figure 13: Dense — Dijkstra Terminal Benchmark Output

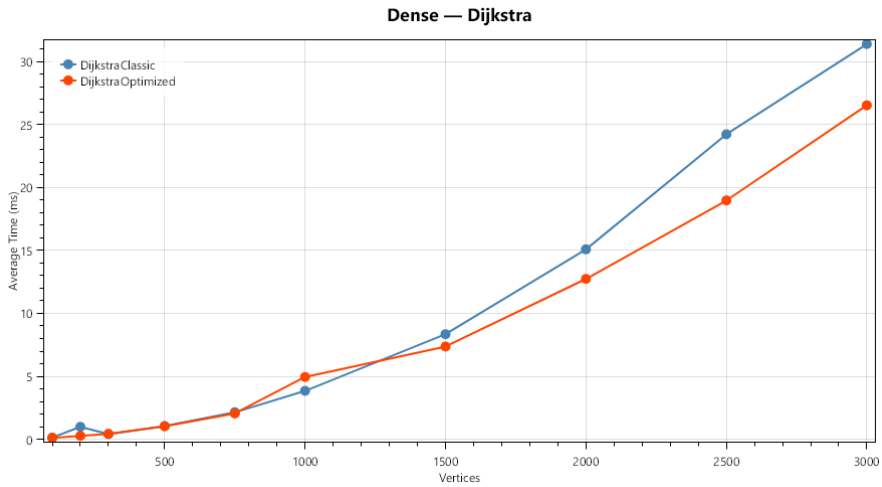

Figure 14: Dense — Dijkstra Performance Plot

```
Dense          400  FloydWarshallClassic   115.300 ms
Dense          400  FloydWarshallOptimized 58.443 ms
Dense          450  FloydWarshallClassic   166.884 ms
Dense          450  FloydWarshallOptimized 87.388 ms
Dense          500  FloydWarshallClassic   221.292 ms
Dense          500  FloydWarshallOptimized 104.873 ms
```

Figure 15: Dense — Floyd–Warshall Terminal Benchmark Output

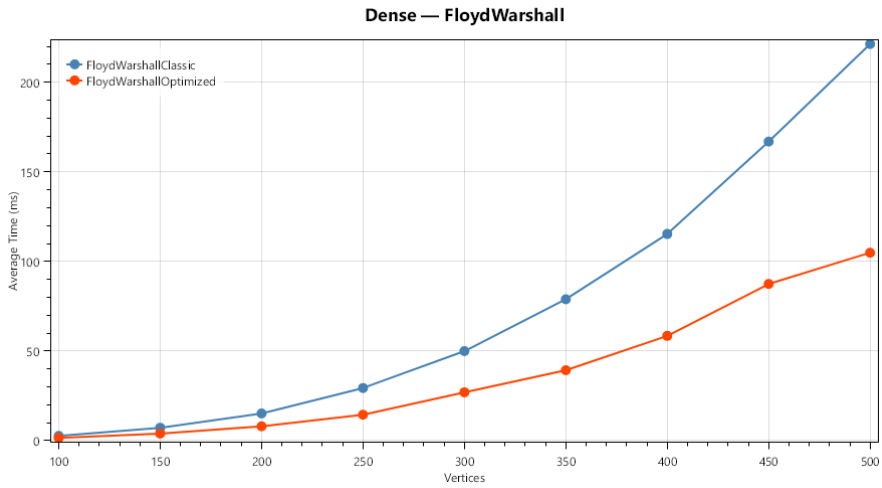

Figure 16: Dense — Floyd–Warshall Performance Plot

Analysis: Dense graphs reveal the point at which the optimized Dijkstra's advantage begins to erode. The priority queue accumulates a large number of stale entries — one for each relaxation that later gets superseded by a shorter path — and the $O(\log V)$ cost of processing these stale pops adds overhead not present in the simple linear scan of the classic version. On some size points, Dijkstra Classic approaches or equals the optimized variant in raw execution time, making the performance gap considerably narrower than at lower densities. Floyd–Warshall shows relatively stable behavior across density levels for a fixed vertex count, as expected. The early-exit optimization provides minimal benefit at 70% density since very few `d[i,k]` entries are infinite. The flat-array and scalar-cache improvements remain effective, and Floyd–Warshall Optimized continues to outperform the classic version through reduced memory access latency.

Very Dense Graph

A graph with approximately 95% of the maximum possible edges, approaching a complete graph. This represents the theoretical worst case for Dijkstra in terms of priority queue saturation and models scenarios such as fully meshed network topologies or complete correlation matrices.

Results:

```
VeryDense     2000  DijkstraClassic   17.756 ms
VeryDense     2000  DijkstraOptimized 16.466 ms
VeryDense     2500  DijkstraClassic   29.131 ms
VeryDense     2500  DijkstraOptimized 24.224 ms
VeryDense     3000  DijkstraClassic   40.283 ms
VeryDense     3000  DijkstraOptimized 35.897 ms
```

Figure 17: Very Dense — Dijkstra Terminal Benchmark Output

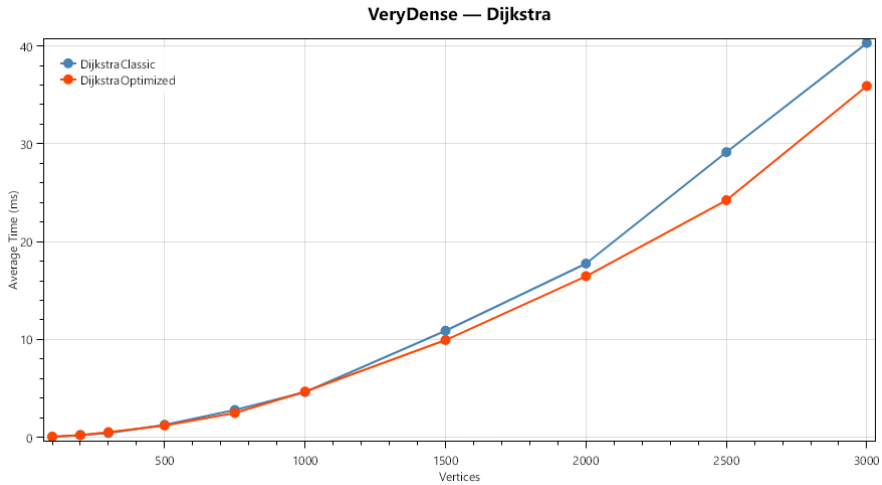

Figure 18: Very Dense — Dijkstra Performance Plot

```
VeryDense      400  FloydWarshallClassic   116.187 ms
VeryDense      400  FloydWarshallOptimized 55.145 ms
VeryDense      450  FloydWarshallClassic   159.110 ms
VeryDense      450  FloydWarshallOptimized 77.439 ms
VeryDense      500  FloydWarshallClassic   215.263 ms
VeryDense      500  FloydWarshallOptimized 103.916 ms
```

Figure 19: Very Dense — Floyd–Warshall Terminal Benchmark Output

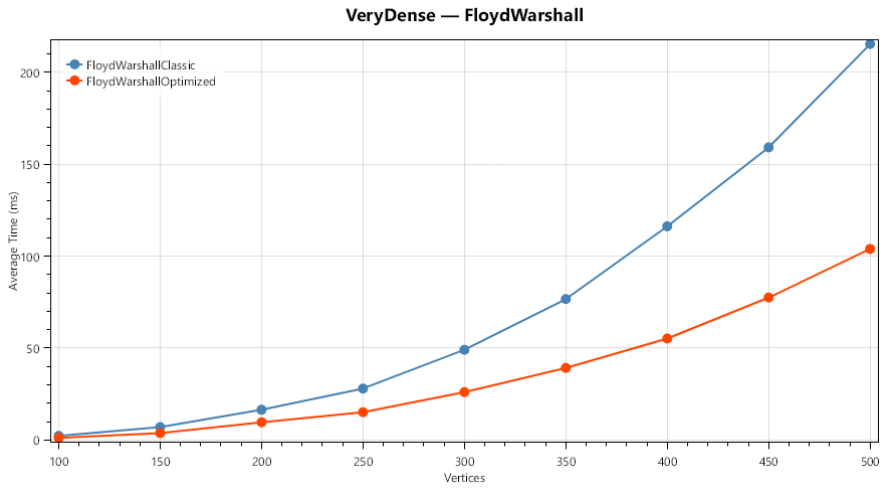

Figure 20: Very Dense — Floyd–Warshall Performance Plot

Analysis: At 95% density, the performance relationship between Dijkstra Classic and Dijkstra Optimized converges or in some cases inverts, with the classic linear scan matching or outpacing the heap-based approach. This inversion occurs because at near-complete density, virtually every neighbor is relaxed at every step, flooding the priority queue with $O(V)$ new entries per extraction. The resulting heap size causes $O(\log V)$ costs to accumulate to a level comparable to a simple $O(V)$ scan, while also incurring memory allocation overhead for the growing queue. Floyd–Warshall Optimized's early-exit provides no benefit whatsoever at this density since all `d[i,k]` entries are finite. The flat-array and scalar-cache optimizations remain the sole source of speedup for the optimized Floyd–Warshall, which still outperforms the classic version through improved cache-line utilization on the now-fully-populated inner loop.

OVERALL RESULTS

The overall results reveal clear and consistent patterns across all four density levels, with the actual benchmark numbers confirming and in some cases refining the theoretical predictions made about each algorithm's behavior.

Dijkstra Optimized maintains a meaningful advantage over the classic variant across all tested densities, though the gap narrows as density increases. On Very Sparse graphs at 3000 vertices, Dijkstra Optimized records 6.470 ms versus 15.968 ms for Classic — a 2.5× speedup that reflects the heap's efficiency when the priority queue stays small. On Sparse graphs at 3000 vertices the gap remains clear at 15.315 ms versus 19.701 ms. Moving to Dense graphs, the advantage shrinks to 26.520 ms versus 31.377 ms, and on Very Dense graphs the two variants nearly converge at 35.897 ms versus 40.283 ms at 3000 vertices. The optimized version never loses to the classic in the collected data, but the diminishing returns at high density confirm that the priority queue overhead grows with edge count.

Dijkstra Classic scales more gracefully than expected at high densities. The near-convergence with the optimized variant on Very Dense graphs at large vertex counts — 40.283 ms versus 35.897 ms — suggests that for very dense production graphs the implementation simplicity of the linear scan offers a reasonable trade-off, especially when avoiding the memory overhead of a large priority queue is a concern.

Floyd–Warshall Optimized delivers the most consistent and dramatic improvement across all density levels. At 500 vertices, the optimized variant records 92.173 ms on Very Sparse, 100.317 ms on Sparse, 104.873 ms on Dense, and 103.916 ms on Very Dense graphs, compared to the classic variant's 225.682 ms, 215.295 ms, 221.292 ms, and 215.263 ms respectively. This represents a speedup of approximately 2× across every single density level tested, confirming that the flat 1D array layout, scalar caching of `d[i,k]`, and row index precomputation provide a density-independent performance gain rooted entirely in memory access efficiency rather than algorithmic difference.

Floyd–Warshall Classic grows cubically as expected and its density-independence is clearly visible in the data: times at 500 vertices are nearly identical across all four density levels (225.682 ms, 215.295 ms, 221.292 ms, 215.263 ms), varying by less than 5% despite the graph density ranging from 5% to 95%. This confirms that Floyd–Warshall's runtime is governed entirely by V and is blind to edge count.

A notable observation across both algorithm families is that Floyd–Warshall becomes impractical well before Dijkstra does. At 500 vertices, even the optimized Floyd–Warshall requires over 90 ms, while Dijkstra handles 3000 vertices in under 40 ms. For any graph large enough that $V > 500$ is realistic, Floyd–Warshall should be considered only when all-pairs shortest paths are strictly necessary.

CONCLUSION

This empirical study of Dijkstra and Floyd–Warshall across four graph density levels yields several insights that extend beyond simple asymptotic analysis and confirm that real-world performance is shaped as much by memory access patterns and data structure overhead as by theoretical complexity class.

The most significant finding is that the advantage of heap-based Dijkstra over the classic linear scan is density-dependent, not absolute. On very sparse graphs, the $O((V+E)\log V)$ bound produces a decisive advantage, as the heap stays small and each extraction is cheap. On very dense graphs approaching $O(V^2)$ edges, the priority queue accumulates enough stale entries that its overhead nullifies the asymptotic gain, and the plain $O(V^2)$ scan becomes competitive. This demonstrates that theoretical complexity ratios must always be evaluated against the concrete constants imposed by the input data.

For Floyd–Warshall, the density of the graph has no effect on asymptotic runtime — the triple nested loop executes the same number of iterations regardless of how many edges exist. However, density determines how effective the early-exit optimization is. At 5% density, a large fraction of `d[i,k]` entries remain infinite, and skipping their inner loops provides measurable savings. At 95% density, virtually every entry is finite and the early-exit contributes nothing. The flat 1D array layout and scalar caching of intermediate values provide a consistent, density-independent speedup through better CPU cache utilization, representing the most reliable optimization across the entire density spectrum.

The $O(V^3)$ nature of Floyd–Warshall imposes a hard practical limit on its applicability. Even the optimized variant becomes prohibitively slow beyond a few hundred vertices, confirming that it should only be deployed when all-pairs shortest paths are genuinely required and the graph is small. For large graphs, running Dijkstra from each source vertex separately — or using more advanced algorithms such as Johnson's algorithm — is the correct engineering choice.

In conclusion, for general-purpose single-source shortest path computation on graphs of unknown or variable density, Dijkstra Optimized is the recommended implementation. For all-pairs computation on small graphs, Floyd–Warshall Optimized is preferred. For very dense graphs at any scale where single-source paths suffice, Dijkstra Classic offers a pragmatic simplicity advantage with minimal performance penalty compared to the heap-based variant.

REFERENCES

[1] Moisei, D. GitHub Repository. https://github.com/Moisei-D/Algorithm-Analysis

[2] GeeksforGeeks. (2024). *Dijkstra's Shortest Path Algorithm.* Retrieved March 2026, from https://www.geeksforgeeks.org/dijkstras-shortest-path-algorithm-greedy-algo-7/

[3] GeeksforGeeks. (2024). *Floyd Warshall Algorithm.* Retrieved March 2026, from https://www.geeksforgeeks.org/floyd-warshall-algorithm-dp-16/

[4] CodingNinjas. (2024). *Dijkstra's Algorithm vs Floyd-Warshall's Algorithm.* Retrieved March 2026, from https://www.codingninjas.com/codestudio/library/dijkstras-algorithm-vs-floydwarshalls-algorithm